

Group Members:

1. Daniel Herrera (Team Leader)
2. Carlos Martinez
3. Anthony Jimenez
4. Harut Kulakchian
5. Osbaldo Bravo

Team APIs

API 1: Register User (Sign Up)

API Name

Register

Purpose

Creates a new user account in the FitQuest system. The API validates input, hashes the password using bcrypt, and creates a new user document with default character stats (Strength: 5, Health: 100, level: 1, XP: 0, gold: 100).

Method

POST

Cloud Service

Railway (Deployed at: <https://fitquest-production.up.railway.app>)

Alternative:

AWS API Gateway + Lambda, Google Cloud Run, Azure Functions

Input

Content-Type	Type	Header	Required	Description
Email	String	Body	Yes	User email address
Password	String	Body	Yes	User password
heroName	String	Body	Yes	User display name
Content-Type	String	Body	Yes	application/json

Sample Input:

```
{
  "email": "player@example.com",
  "password": "SecurePass123",
  "heroName": "SquatMaster"
}
```

Output Success Response (200 OK):

```
{
  "success": true,
  "message": "User registered"
}
```

Error Responses:

Status Code	Response
400 Bad Request	{"success": false, "message": "Boss has already been defeated"}
500 Internal Error	{"success": false, "message": "Server error"}

Related Module
Authentication Module

Related Database Table(s)

Table/Collection	Operation	Description
Users	INSERT	Creates new user with email, hashed password, heroName, and default stats

User Document Structure:

```
{
  "_id": ObjectId("..."),
  "email": "player@example.com",
  "password": "$2b$10$hashedpassword...",
  "heroName": "SquatMaster",
  "stats": {
    "strength": 5,
    "stamina": 5,
    "agility": 5,
    "health": 100
  },
  "level": 1,
  "xp": 0,
  "gold": 100,
  "waterIntake": [],
  "workouts": [],
  "battles": [],
  "activeQuests": [],
  "inventory": []
}
```

API 2: Login User

API Name

Login User

Purpose

Authenticates an existing user by verifying email and password. On successful authentication, generates and returns a JSON Web Token (JWT) for session management, along with the user's character data (heroName, stats, level, gold).

Method

POST

Cloud Service

Railway (Deployed at: <https://fitquest-production.up.railway.app>)

Alternative:

AWS API Gateway + Lambda, Google Cloud Run, Azure Functions

Input

Parameter	Type	Location	Required	Description
email	string	Body	Yes	User's registered email address
password	string	Body	Yes	User's password
Content-Type	string	Header	Yes	application/json

Sample Input:

```
{
  "email": "player@example.com",
  "password": "SecurePass123"
}
```

Output

Success Response (200 OK):

json

```
{
  "success": true,
  "message": "Login successful",
  "token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWFpbCI6InBsYXllckBleGFtcGxlLnNvbSIsInVzZXJJZCI6IjY3ZjNmMjhhMjRjZmRiOWMxNmMzNDU2IiwiaWF0IjoxNzQ0MjAwMDAwLCJleHAiOiE3NDQyMDcyMDB9.xyz123",
  "heroName": "SquatMaster",
  "stats": {
    "strength": 7,
    "stamina": 5,
    "agility": 5,
    "health": 100
  }
}
```

```
},  
  "level": 2,  
  "gold": 150  
}
```

Error Responses:

Status Code	Response
400 Bad Request	{"success": false, "message": "User not found"}
400 Bad Request	{"success": false, "message": "Incorrect password"}
500 Internal Error	{"success": false, "message": "Server error"}
Related Module	
Authentication Module	

Related Database Table(s)

Table/Collection	Operation	Description
Users	SELECT	Finds user by email to verify credentials and retrieve character data

Query Used:

```
db.users.findOne({ email: "player@example.com" })
```

Fields Retrieved:

- password (for bcrypt comparison)
- heroName
- stats
- level
- gold
- _id (for JWT token)

Peer API Testing (In-Class Activity)

Team Tested:

Team Asteroid Destroyer

APIs Tested:

Text Summarization API

What is good:

- The API is easy to use and clearly to understand
- The input fields (text and mode) are simple and understandable
- The API returns a response

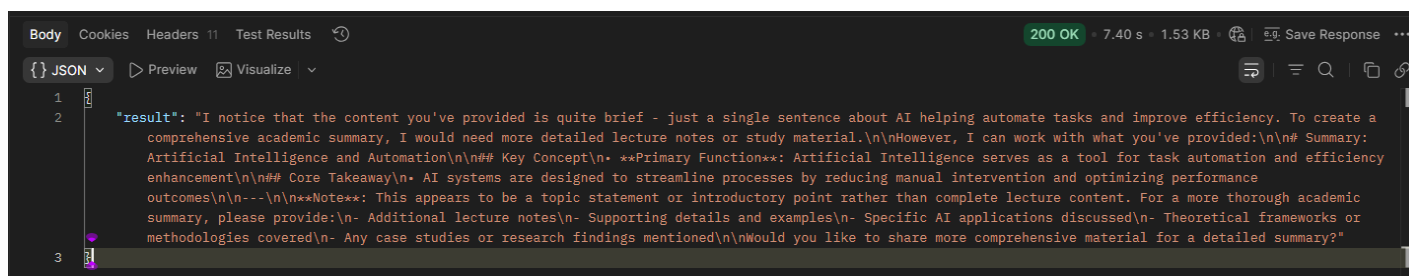
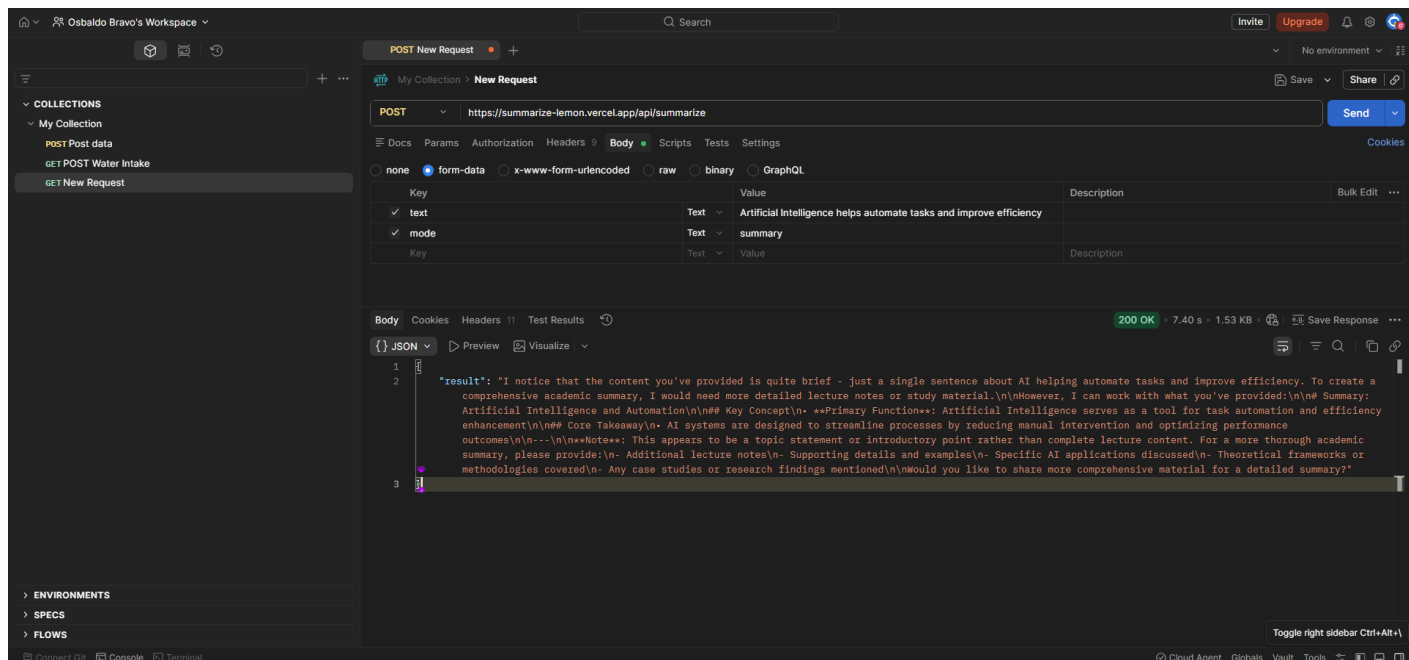
- The output successfully summarizes the input text

Issues found:

- The response uses the key “result” instead of something clearer like “summary”
- There is no validation when the text input is empty
- There is no clear error message when an invalid mode is entered

Suggestions for improvement:

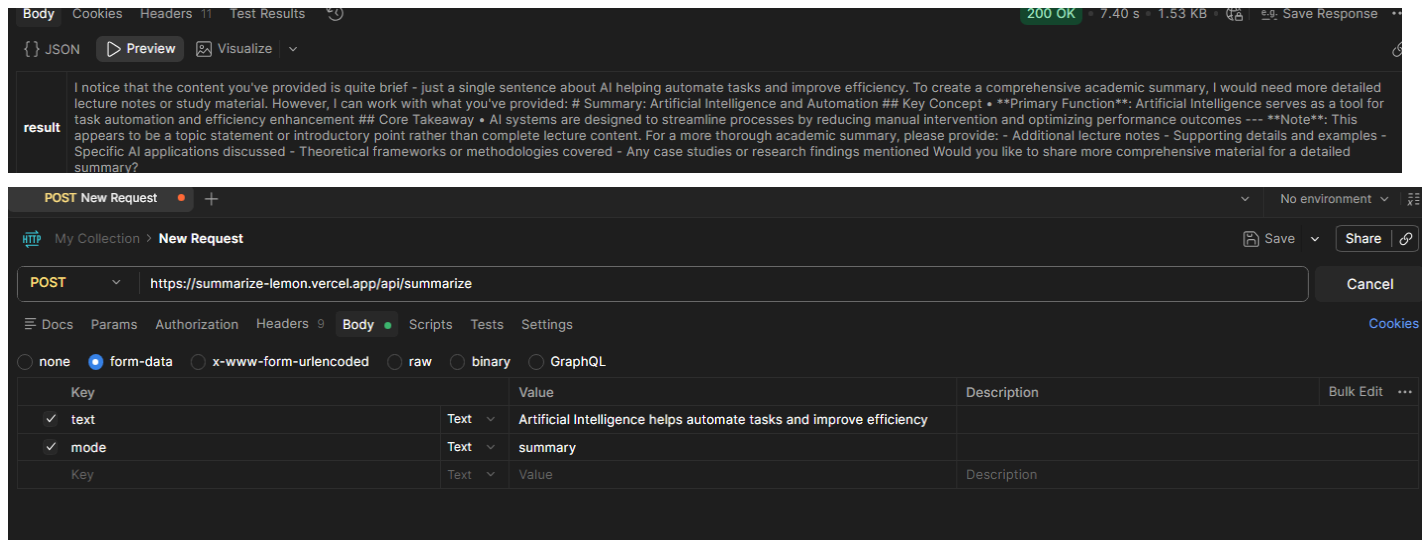
- Change “result” to “summary” for better clarity
- Add error handling for empty text input



("result": "I notice that the content you've provided is quite brief - just a single sentence about AI helping automate tasks and improve efficiency. To create a comprehensive academic summary, I would need more detailed lecture notes or study material. However, I can work with what you've provided: Summary: Artificial Intelligence and Automation Key Concept Primary Function: Artificial Intelligence serves as a tool for task automation and efficiency enhancement Core Takeaway AI systems are designed to streamline processes by reducing manual intervention and optimizing performance outcomes Note: This

appears to be a topic statement or introductory point rather than complete lecture content. For a more thorough academic summary, please provide:\n- Additional lecture notes\n- Supporting details and examples\n- Specific AI applications discussed\n- Theoretical frameworks or methodologies covered\n- Any case studies or research findings mentioned\n\nWould you like to share more comprehensive material for a detailed summary?"

}}



Post - <https://summarize-lemon.vercel.app/api/summarize>

Summary:

The API works well and returns summarized text based on user input. It is simple to use and responds quickly. Testing with Postman provided a clear view of how the API processes requests and returns results.